WEB ARCHITECTURE LAB 02

Servlets, Sessions, JSP, User Auth & Cart Management

Building a Cart Management

Prepared by:
Njayou Youssouf
Lecturer, IUT CSE

Department of Computer Science and Engineering
Islamic University of Technology
May 21, 2026

Lab Overview

This lab takes you on a journey from writing your very first servlet to building a **multi-page shopping cart application** with user authentication — all using Java Servlets, JSP, and HttpSession.

The lab is divided into **two parts**. **Part I (Tasks 1–3)** covers servlet fundamentals: your first servlet, request parameters, and session tracking. **Part II (Tasks 4–6)** builds on those foundations to create a complete e-commerce-style application with login, product browsing, and cart management.

Lab Objectives

By the end of this lab, you will be able to:

- Write HttpServlets that handle GET and POST requests with form parameters.
- Map servlets to URL patterns using web.xml or use annotations.
- Use the HttpSession API to maintain state across stateless HTTP requests.
- Implement session-based user registration, login, and logout.
- Build JSP pages that display dynamic content using scriptlets and expressions.
- Create a JavaBean to encapsulate shopping cart logic (the Model in MVC).
- Coordinate multiple servlets and JSPs into a cohesive multi-page web application.

Prerequisites

For Part I: **IntelliJ IDEA**, **JDK 8+**, **Apache Tomcat 9.x**, and familiarity with HTML forms and HTTP (Lectures 01–02).

For Part II: everything above, plus basic understanding of **JSP syntax** (scriptlets, expressions, implicit objects) and **Java collections** (HashMap).

> **Quick Recap**
> *HTTP is stateless — the server forgets you after each request. Servlets intercept requests and generate responses inside a container like Tomcat. HttpSession lets you store data that persists across requests. JSP lets you write HTML with embedded Java instead of writing HTML inside println() calls.*

Project Setup (All Tasks)

All six tasks share a single Maven web application project.

Step 1: Create the Project

In IntelliJ: **File → New → Project → Maven Archetype**, select `maven-archetype-webapp`. Name it **`WebArchiLab02`**.

Step 2: Directory Structure

```
WebArchiLab02/
├── pom.xml
└── src/main/
    ├── java/              ← Servlet and Bean classes
    └── webapp/
        ├── index.html        ← Tasks 1–3 front-end
        ├── login.jsp          ← Task 4 login/register page
        ├── homepage.jsp       ← Task 5 product catalog
        ├── cart.jsp           ← Task 6 cart management
        └── WEB-INF/
            └── web.xml        ← Deployment descriptor
```

Step 3: pom.xml Dependency

```xml
<dependency>
    <groupId>javax</groupId>
    <artifactId>javaee-web-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
</dependency>
```

Step 4: Configure Tomcat

Run → Edit Configurations → Tomcat Server → Local. Deploy the WAR artifact with context `/WebArchiLab02`.

PART I

Servlet Fundamentals

Tasks 1–3: Your first servlet, request parameters, and session tracking

TASK 1: Hello Servlet — Guided Walkthrough

Worked Example

This task is your reference example. Study it carefully — you'll use the same patterns (HTML form → servlet → web.xml mapping → response) to complete all remaining tasks on your own.

You'll create an HTML form that asks for a name, submit it to a servlet, and display a personalized greeting.

1.1 The HTML Form

Create `index.html` in `webapp/`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Web Architecture Lab 02</title>
</head>
<body>
    <h1>Task 1: Hello Servlet</h1>
    <p>Please enter your name below:</p>
    <form name="myform" method="get" action="myServlet">
        <input type="text" name="user" />
        <br><br>
        <input type="submit" value="Submit" />
    </form>
</body>
</html>
```

Key points: `action="myServlet"` tells the browser which URL to hit. `method="get"` appends data as query parameters. `name="user"` becomes the parameter key the servlet reads.

1.2 The Servlet Class

Create `myServlet.java` in `src/main/java/`:

```java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

public class myServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        String userName = request.getParameter("user");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HTML><HEAD><TITLE>Response</TITLE></HEAD>");
        out.println("<BODY><h2>Hello, " + userName + "!</h2>");
        out.println("</BODY></HTML>");
    }
}
```

Anatomy of a Servlet

Every servlet extends HttpServlet and overrides doGet() or doPost(). Tomcat calls these when a matching request arrives. The request object gives you parameters and session access. The response object lets you write output.

1.3 web.xml Configuration

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         version="4.0">

    <servlet>
        <servlet-name>myServlet</servlet-name>
        <servlet-class>myServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>myServlet</servlet-name>
        <url-pattern>/myServlet</url-pattern>
    </servlet-mapping>
</web-app>
```

Two-step pattern: `<servlet>` registers the class, `<servlet-mapping>` binds it to a URL. When someone hits `/myServlet`, Tomcat finds the class and calls `doGet()`.

This is the pattern you'll reuse everywhere: **(1)** HTML form with action URL, **(2)** servlet class with doGet/doPost, **(3)** web.xml entries to wire URL → class.

TASK 2: Random Number Generator Servlet

Requirements

Build a servlet that takes two numbers as input and displays a random number between them.

Component	Details
HTML Form	Two number inputs (lower/upper bound) + Submit button. Add to index.html.
Form Method	GET
Servlet	randomNumberServlet.java
URL Pattern	/randomNumberServlet
Behavior	Read both numbers, generate a random integer in [lower, upper), display in an <h1> tag.

Expected Output

If the user enters 10 and 50, something like: **Random Number: 37**

Hints

java.util.Random.nextInt(bound) generates 0 to bound-1. Form values arrive as Strings — use Integer.parseInt(). Don't forget the web.xml entries.

TASK 3: Session Tracking with HttpSession

Background

HTTP is ***stateless*** — the server forgets you after each request. Java's **`HttpSession`** API solves this by associating a server-side session object with each client via a `JSESSIONID` cookie.

API Reference

Method	What It Does
`request.getSession()`	Returns current session or creates one.
`request.getSession(false)`	Returns current session or null (no creation).
`session.getId()`	Returns unique session ID string.
`session.getCreationTime()`	When session was created (ms since epoch).
`session.getLastAccessedTime()`	Last client request time in this session.
`session.invalidate()`	Destroys the session.

Requirements

Build two servlets:

`httpSessionServlet` (POST, mapped to `/httpSessionServlet`):

1. Check if a session exists (without creating one).
2. No session: create one, set visit count = 0, display **"Welcome, Newcomer"**
3. Session exists: increment count, display **"Welcome back!"**
4. Display table with: Session ID, Creation Time, Last Access Time, Previous Accesses.
5. Include an "End Session" button targeting the second servlet.

`endSessionServlet` (GET, mapped to `/endSessionServlet`):

1. Retrieve session without creating. If exists, invalidate it.
2. Redirect to `index.html`.

Expected Behavior

Scenario	What You Should See

First click	"Welcome, Newcomer". Session info table. Previous Accesses = 0.
Second click	"Welcome back!". Same Session ID. Previous Accesses = 1.
After End Session	Redirects to index.html. Next click = new session, newcomer again.

Hints

Use getSession(false) to check without creating. For timestamps: new java.util.Date(session.getCreationTime()).toString(). For redirect: response.sendRedirect("index.html").

P A R T I I

Shopping Cart Application

Tasks 4–6: User authentication, product catalog, and cart management using Servlets, JSP & HttpSession

In Part I you learned the building blocks: servlets, request parameters, and sessions. Now you'll combine them into a **real multi-page web application**. Think of it as a tiny version of the backend that powers sites like Daraz or Amazon — registration, login, product browsing, and cart management, all wired together with session data.

Why JSP?

In Task 1–3, you wrote HTML inside println() calls — painful for anything beyond a few lines. JSP flips this: you write HTML and embed Java where needed using <% %> scriptlets and <%= %> expressions. The server compiles each JSP into a servlet behind the scenes.

Key Design Decision: Session-Only Storage

To keep things simple, **all data lives in the session only**. Registered users are stored in a `HashMap<String, String>` in the `ServletContext` (application scope). This means: if you restart the server, all users are gone and everyone must re-register. No database, no XML files — just Java objects in memory.

Storing passwords in plain text and in memory is for learning purposes only. In production you would hash passwords and use a database.

Project Components for Part II

File	Type	Responsibility
`LoginServlet.java`	Servlet	Handles registration, login, and credential checking
`LogoutServlet.java`	Servlet	Invalidates session, redirects to login
`CartServlet.java`	Servlet	Processes add/increment/decrement cart actions
`CartBean.java`	JavaBean	Encapsulates cart state (items + quantities)
`login.jsp`	JSP	Login and registration form
`homepage.jsp`	JSP	Product catalog with Add to Cart buttons
`cart.jsp`	JSP	Cart view with +/– quantity controls

TASK 4: User Authentication (Register, Login, Logout)

YOUR TASK

Build a complete login/registration system using session-based storage. No database or XML is needed — user data lives in application-scoped memory only.

Requirements

A) login.jsp — the entry point of your application:

1. Two text fields: username and password.
2. Two submit buttons: "Login" and "Register" (both POST to LoginServlet).
3. Display an error message if login fails or registration has a conflict (passed via request parameter or attribute).

B) LoginServlet.java — handles both login and registration:

1. Determine the action (login vs register) from a hidden field or the button's `name` attribute.
2. For **Registration**: check if username already exists in the application-scoped user map. If not, add it. Create a session and store the username. Redirect to homepage.jsp.
3. For **Login**: verify username exists and password matches. If valid, create session and redirect to homepage.jsp. If invalid, redirect back to login.jsp with an error.

C) LogoutServlet.java — simple:

- Invalidate the session and redirect to login.jsp.

Where to Store Users

Use `ServletContext` (application scope) to store a shared `HashMap<String, String>` mapping usernames to passwords. This is accessible to all servlets and survives across individual sessions, but resets when the server restarts.

```
// In your servlet, access application-scoped data like this:
ServletContext ctx = getServletContext();
HashMap<String, String> users = (HashMap) ctx.getAttribute("users");
if (users == null) {
    users = new HashMap<>();
    ctx.setAttribute("users", users);
}
```

Expected Behavior

Scenario	Result
Register new user	User added to map. Session created with username stored. Redirect to homepage.jsp.
Register existing username	Error message on login.jsp: "Username already taken."
Login with correct credentials	Session created. Redirect to homepage.jsp.
Login with wrong password	Error message: "Invalid credentials."
Login with non-existent user	Error message: "User not found. Please register."
Logout	Session invalidated. Redirect to login.jsp.
Restart server	All users gone — must re-register. This is expected.

Hints

To distinguish Login vs Register from the same form, give each submit button a different name attribute (e.g., name="login" and name="register"). In the servlet, check which parameter is non-null: request.getParameter("login") != null. For session storage: session.setAttribute("username", username). For error messages: response.sendRedirect("login.jsp?error=User+not+found") and read it in JSP with request.getParameter("error").

TASK 5: Product Listing & Adding to Cart

YOUR TASK

Build the product catalog page and implement the "Add to Cart" functionality using a CartBean stored in the session.

Requirements

A) CartBean.java — a plain Java class (JavaBean) that encapsulates your cart:

- Internally uses a `HashMap<String, Integer>` mapping product names to quantities.
- Methods: addItem(name), incrementItem(name), decrementItem(name), getItems(), getTotalCount(), getTotalPrice().
- For pricing, hardcode a price map inside the bean (e.g., `"Wireless Mouse" → 25`) or accept price as a parameter.

B) homepage.jsp — the product catalog:

1. Display a welcome message with the logged-in username (from session).
2. Show at least 3 hardcoded products, each with a name, price, and an "Add to Cart" button.

3. Each "Add to Cart" button sends a GET request to `CartServlet?action=add&item=ProductName`.
4. Display the current total number of items in the cart (from the CartBean in session).
5. Include links to "View Cart" (cart.jsp) and "Logout" (LogoutServlet).

C) CartServlet.java — the controller:

- Reads the `action` and `item` parameters from the request.
- Retrieves the CartBean from the session (or creates one if it doesn't exist).
- Calls the appropriate CartBean method based on the action.
- Redirects back to the referring page (homepage.jsp or cart.jsp).

Make It Realistic

Give your products actual names and prices instead of 'Item 0', 'Item 1'. Think: 'Wireless Mouse — $25', 'Mechanical Keyboard — $75', 'USB-C Hub — $40'.

Expected Behavior

Action	Result
Open homepage	"Welcome, [username]!" with 3+ products listed, each with Add button. Cart count shows 0.
Click "Add to Cart"	Page reloads. Cart count increments by 1.
Add same item twice	Cart count = 2. Internally, that item's quantity = 2.
Navigate to cart and back	Cart count persists — session keeps the data.

Hints

Store the CartBean in the session: session.setAttribute("cart", cartBean). In homepage.jsp, retrieve it: CartBean cart = (CartBean) session.getAttribute("cart"). If null, the count is 0. For the Add button, a simple link works: Add to Cart. After processing, redirect back: response.sendRedirect("homepage.jsp").

TASK 6: Cart View & Quantity Management

YOUR TASK

Build the cart detail page with full quantity controls. This is the most complex task — it ties together JSP, servlet, session, and the CartBean.

Requirements

cart.jsp should display:

- A table listing each item in the cart: product name, unit price, quantity, and subtotal (price × quantity).
- For each item, a "+" button and a "–" button to adjust the quantity.
- Only items with quantity > 0 should appear in the table.
- A total price row at the bottom summing all subtotals.
- A "Back to Shop" link to homepage.jsp.
- A "Logout" link to LogoutServlet.

Button behavior (handled by CartServlet):

- "+" sends `CartServlet?action=increment&item=Name`
- "–" sends `CartServlet?action=decrement&item=Name`
- The – button must not allow quantities below zero. If quantity reaches 0, the item disappears from the cart.
- After processing, CartServlet redirects back to `cart.jsp` (not homepage).

Critical Requirement: Session Persistence

The cart must persist across page navigations. Going to homepage.jsp and back to cart.jsp should show **the same items with the same quantities**. This is only possible if the CartBean stays in the session and both pages read from it.

Hints

In cart.jsp, iterate over the CartBean's items using a scriptlet: <% for (Map.Entry<String, Integer> entry : cart.getItems().entrySet()) { %> then build an HTML table row for each. The + and – buttons can be simple <a> links. In CartServlet, check the action parameter and use a referer header or an extra parameter to decide whether to redirect to homepage.jsp or cart.jsp.